

STRIPE PROJECT

1. OLTP Data System

Annexes

1.A. OLTP Conceptual Data Model (CDM)

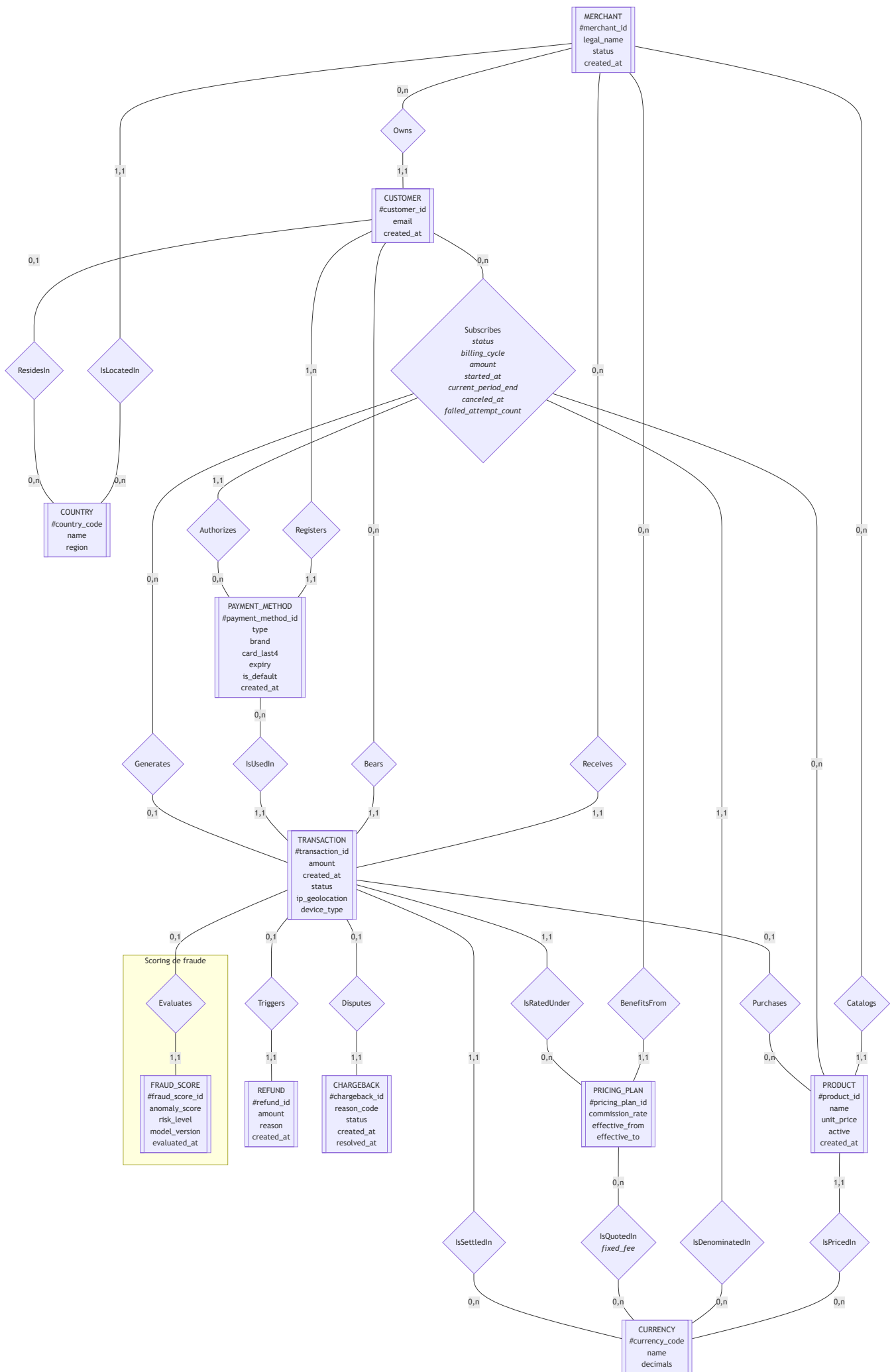
Nicolas Pichon - AIA RNCP 38777 / BC02 / OLTP Data System - Annexe A / v25 - 2026/10/13.

Ce document présente le modèle conceptuel du modèle de données transactionnelles *Stripe*. Il applique le formalisme *Merise* pour découvrir le système conceptuel entités-associations et le transformer en modèle logique. *Merise* décrit d'abord le métier indépendamment de toute implémentation : pas de clé étrangère, pas de typage SQL, pas de table de jointure. Ces éléments n'entrent en jeu qu'au passage au *modèle logique* (cf. §1.A.5).

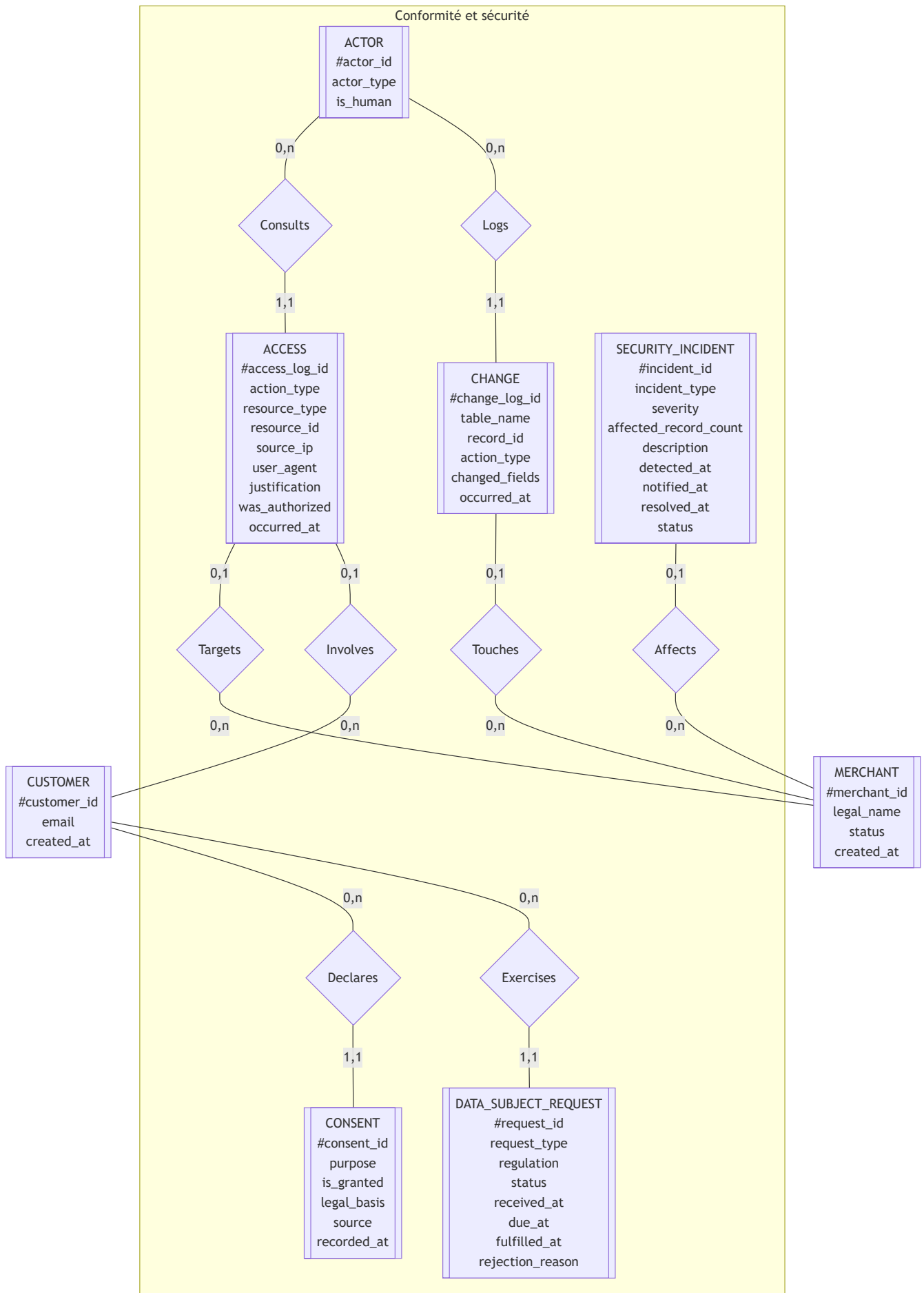
1.A.1. Diagrammes

Le modèle conceptuel est schématisé par les diagrammes suivants (modèle métier, modèle sécurité et conformité). Les rectangles représentent des **entités** et les losanges des **associations**. Les cardinalités sont inscrites sur les liens au format $(\min, \max)$.

1.A.1.1. Modèle *Métier*



1.A.1.2. Modèle Sécurité & Conformité



1.A.2. Dictionnaire des entités

Le symbole # marque l'identifiant.

Entité	Propriétés
COUNTRY	#country_code, name, region
CURRENCY	#currency_code, name, decimals
MERCHANT	#merchant_id, legal_name, status, created_at
CUSTOMER	#customer_id, email, created_at
PAYMENT_METHOD	#payment_method_id, type, brand, card_last4, expiry, is_default, created_at
PRODUCT	#product_id, name, unit_price, active, created_at
TRANSACTION	#transaction_id, amount, created_at, status, ip_geolocation, device_type
REFUND	#refund_id, amount, reason, created_at
CHARGEBACK	#chargeback_id, reason_code, status, created_at, resolved_at
FRAUD_SCORE	#fraud_score_id, anomaly_score, risk_level, model_version, evaluated_at
PRICING_PLAN	#pricing_plan_id, commission_rate, effective_from, effective_to
ACTOR	#actor_id, actor_type, is_human
ACCESS	#access_log_id, action_type, resource_type, resource_id, source_ip, user_agent, justification, was_authorized, occurred_at
CHANGE	#change_log_id, table_name, record_id, action_type, changed_fields, occurred_at
CONSENT	#consent_id, purpose, is_granted, legal_basis, source, recorded_at
DATA_SUBJECT_REQUEST	#request_id, request_type, regulation, status, received_at, due_at, fulfilled_at, rejection_reason
SECURITY_INCIDENT	#incident_id, incident_type, severity, affected_record_count, description, detected_at, notified_at, resolved_at, status

1.A.3. Dictionnaire des associations

Association	Entité A	Card. A	Entité B	Card. B	Propriétés portées
IsLocatedIn	MERCHANT	1,1	COUNTRY	0,n	-
ResidesIn	CUSTOMER	0,1	COUNTRY	0,n	-
Owns	MERCHANT	0,n	CUSTOMER	1,1	-
Catalogs	MERCHANT	0,n	PRODUCT	1,1	-
IsPricedIn	PRODUCT	1,1	CURRENCY	0,n	-
Registers	CUSTOMER	1,n	PAYMENT_METHOD	1,1	-
Subscribes	CUSTOMER	0,n	PRODUCT	0,n	status , billing_cycle , amount , started_at , current_period_end , canceled_at , failed_attempt_count
Authorizes	SUBSCRIBES	1,1	PAYMENT_METHOD	0,n	-
IsDenominatedIn (ajouté v22)	SUBSCRIBES	1,1	CURRENCY	0,n	-
BenefitsFrom	MERCHANT	0,n	PRICING_PLAN	1,1	-
IsQuotedIn	PRICING_PLAN	0,n	CURRENCY	0,n	fixed_fee
Receives	MERCHANT	0,n	TRANSACTION	1,1	-
Bears	CUSTOMER	0,n	TRANSACTION	1,1	-
IsUsedIn	PAYMENT_METHOD	0,n	TRANSACTION	1,1	-
IsSettledIn	TRANSACTION	1,1	CURRENCY	0,n	-
IsRatedUnder	TRANSACTION	1,1	PRICING_PLAN	0,n	-
Generates	SUBSCRIBES	0,n	TRANSACTION	0,1	-
Purchases	TRANSACTION	0,1	PRODUCT	0,n	-
Triggers	TRANSACTION	0,1	REFUND	1,1	-
Disputes	TRANSACTION	0,1	CHARGEBACK	1,1	-
Evaluates	TRANSACTION	0,1	FRAUD_SCORE	1,1	-
Consults	ACTOR	0,n	ACCESS	1,1	-
Logs	ACTOR	0,n	CHANGE	1,1	-
Targets	ACCESS	0,1	MERCHANT	0,n	-
Involves	ACCESS	0,1	CUSTOMER	0,n	-
Touches	CHANGE	0,1	MERCHANT	0,n	-
Declares	CUSTOMER	0,n	CONSENT	1,1	-
Exercises	CUSTOMER	0,n	DATA_SUBJECT_REQUEST	1,1	-
Affects	SECURITY_INCIDENT	0,1	MERCHANT	0,n	-

1.A.4. Règles de gestion

Les règles de métier que le modèle de conception formalise :

1. Un *marchand* est établi dans un et un seul *pays* ; un *pays* peut accueillir plusieurs *marchands*, ou aucun.
2. Un *client* appartient à un et un seul *marchand*. Une même personne physique achetant chez plusieurs *marchands* correspond à plusieurs occurrences de *CUSTOMER* : le modèle conceptuel modélise les relations *client-marchand* et non les individus.
3. Un *client* enregistre au moins un *moyen de paiement* ; un *moyen de paiement* appartient à un seul *client*.
4. Un *produit* appartient au catalogue d'un seul *marchand* et est tarifé dans une seule *devise*.
5. Un *client* peut souscrire plusieurs *produits*, et un *produit* peut être souscrit par plusieurs *clients* : l'association *Subscribes* est de type *many-to-many* et porte ses propres propriétés. L'*abonnement* est un service du *marchand* ; *Stripe* en fournit l'infrastructure.
6. Une *transaction* peut porter sur un *produit* du catalogue, ou être un paiement libre. Elle peut résulter d'une souscription (prélèvement récurrent) ou être ponctuelle.
7. Une *transaction* est reçue par un seul *marchand*, concerne un seul *client*, réglée par un seul *moyen de paiement*, libellée dans une seule *devise*.
8. Une *transaction* donne lieu à un *remboursement* au plus, un *litige* au plus, et à une *évaluation de fraude* au plus.
9. Un *remboursement*, un *litige* et un *score de fraude* se rapportent obligatoirement à une *transaction* : ce sont des entités **dépendantes**.
10. Un *marchand* peut exister avant d'avoir un *client*, un *produit* catalogué, une *transaction* reçue ou un *plan tarifaire* en vigueur (cas d'un marchand nouvellement onboardé, en cours de vérification KYB). De même, un *pays* ou une *devise* peuvent figurer dans le référentiel sans qu'aucun *marchand*, *produit* ou *transaction* ne les utilise encore : ce sont des données de référence, dont l'existence est indépendante de leur usage.
11. Une *transaction* reliée à une occurrence de *Generates* résulte d'un prélèvement automatique déclenché par le *marchand* dans le cadre d'un *abonnement* ; en l'absence d'un tel lien, la transaction résulte d'une action directe du *client* (achat catalogue via *Purchases*, ou paiement libre). Ces différents types d'initiative ne sont pas stockés comme attribut au niveau conceptuel : ils se déduisent de la présence ou non du lien à *Generates*. Cette règle couvre entièrement l'attribut *Transaction.is_merchant_initiated* introduit au MLD : aucun attribut conceptuel supplémentaire n'est nécessaire, la donnée logique est un booléen dérivé de ce lien.
12. Une *souscription* est obligatoirement autorisée sur un seul *moyen de paiement* (celui qui sera débité à chaque échéance) ; un même *moyen de paiement* peut autoriser plusieurs *souscriptions*, ou aucune. Le *moyen de paiement* autorisé pour une *souscription* appartient obligatoirement au même *client* que celui qui a souscrit : c'est une contrainte de cohérence entre deux chemins du modèle (*Subscribes* → *Customer* d'une part, *Authorizes* → *PaymentMethod* → *Registers* → *Customer* d'autre part), que les cardinalités seules ne peuvent pas exprimer. Elle devra être vérifiée par une contrainte applicative ou un contrôle d'intégrité au niveau logique/physique (par exemple une contrainte *CHECK* /trigger comparant *Subscription.customer_id* et *PaymentMethod.customer_id*).
13. *canceled_at* et *current_period_end* sont deux informations indépendantes et ne doivent pas être confondues : la première marque le moment où le *client* a demandé l'arrêt de l'*abonnement*, la seconde marque la fin de la période déjà facturée. Une *souscription* résiliée reste *active* (au sens du *status*) jusqu'à *current_period_end* : le *client* continue de bénéficier du service qu'il a déjà payé, même après avoir demandé la résiliation. *canceled_at* reste vide tant qu'aucune demande de résiliation n'a été faite, y compris pour une *souscription* qui prend fin par un autre mécanisme (échec de prélèvement répété, par exemple).
14. Un *marchand* peut bénéficier de plusieurs *plans tarifaires* successifs au fil du temps (renégociation de commission, changement de catégorie de risque...) ; à une date donnée, un seul plan est en vigueur pour un marchand donné. Cette absence de chevauchement entre les périodes de validité (*effective_from* / *effective_to*) de deux plans d'un même marchand n'est pas exprimable par les cardinalités seules : elle devra être vérifiée par une contrainte au niveau logique/physique.
15. Le *plan tarifaire* réellement appliqué à une *transaction* (*IsRatedUnder*) est théoriquement dérivable du marchand récepteur et de la date de la transaction (celui dont la période de validité couvre *created_at*) ; il est néanmoins rendu explicite par une association dédiée, pour garantir la traçabilité historique du calcul de la commission - y compris si un plan tarifaire est corrigé rétroactivement. C'est le même principe de conception que celui déjà retenu côté OLAP pour le taux de change de référence (*DimReferenceExchangeRate*).
16. Le *plan tarifaire* facturé à une transaction (*IsRatedUnder*) doit obligatoirement être un plan dont bénéficie le marchand qui a reçu cette transaction (*Receives* + *BenefitsFrom*) : contrainte de cohérence entre deux chemins du modèle, du même type que celle de la règle #12, à vérifier par une contrainte d'intégrité au niveau logique/physique plutôt que par les

- cardinalités. Le montant de commission lui-même (`amount` de `Transaction` × `commission_rate` du plan facturé + `fixed_fee` porté par `IsQuotedIn` pour la devise de cette transaction) n'est **pas stocké comme attribut** au niveau conceptuel : c'est une valeur dérivée, dans le même esprit que la règle #11 pour `is_merchant_initiated`.
17. Un *acteur* (utilisateur, service, système, ou API d'un marchand) peut ne réaliser aucun accès ni aucun changement - cas d'un acteur nouvellement créé - ou en réaliser plusieurs. Un accès ou un changement est en revanche toujours réalisé par un seul acteur identifié. L'identité de l'acteur (`actor_id`) est gérée par un système d'identité externe au périmètre de ce modèle : `ACTOR` en représente la trace dans le domaine de l'audit, pas la source de vérité de l'identité elle-même.
 18. Un *accès* peut concerner un *marchand* donné (cloisonnement des données) et/ou un client donné (pour répondre à une demande d'accès RGPD), mais aucun des deux n'est obligatoire : un accès d'administration transverse à la plateforme ne concerne ni l'un ni l'autre.
 19. Un *changement* porte sur un enregistrement d'une table quelconque du système, désignée par son nom et l'identifiant de l'enregistrement (`table_name` , `record_id`) - sans lien structurel vers cette entité, puisque la nature de la table modifiée n'est pas fixée à l'avance et peut concerner n'importe quelle partie du modèle. Un changement concerne au plus un marchand ; il ne porte jamais sur un client de façon isolée.
 20. Un *consentement* et une *demande d'exercice de droit* se rapportent chacun à un seul client. Le marchand concerné n'apparaît **pas** comme lien conceptuel distinct : contrairement au plan tarifaire (qui varie dans le temps) ou au moyen de paiement autorisé (qui peut différer d'une souscription à l'autre), le marchand d'un client est fixe et entièrement déductible via `owns` (règle #2). Il pourra néanmoins être matérialisé comme colonne dénormalisée au MLD, pour éviter une jointure systématique dans les rapports de conformité - même raisonnement que pour `is_merchant_initiated` (règle #11).
 21. Un *consentement*, une fois enregistré, n'est jamais modifié : un retrait de consentement crée une nouvelle occurrence de `CONSENT` , il n'écrase pas la précédente (historique append-only). L'état en vigueur pour un couple (client, finalité) donné se lit sur l'occurrence la plus récente.
 22. Un *incident de sécurité* peut concerner un marchand précis ou être transverse à la plateforme (`Affects` en 0,1) ; il ne se rapporte à aucun client individuel dans ce modèle - `affected_record_count` reste une grandeur descriptive, sans lien vers les enregistrements concernés eux-mêmes.
 23. `ACCESS` et `CHANGE` sont des journaux en écriture seule au sens métier : une occurrence, une fois créée, n'est jamais modifiée ni supprimée. C'est la condition de validité d'un journal d'audit. A l'inverse, `DATA_SUBJECT_REQUEST` et `SECURITY_INCIDENT` sont des entités de workflow mutables : leur `status` évolue dans le temps (une demande passe de `received` à `fulfilled` , un incident de `open` à `closed`), et des dates de traitement viennent compléter l'enregistrement initial. Cette distinction n'est pas exprimable par les cardinalités ; elle devra être portée par les droits d'accès au niveau physique (privilèges `UPDATE/DELETE` restreints pour `ACCESS` et `CHANGE` uniquement).
 24. `Targets` et `Involves` sont deux liens indépendants et tous deux optionnels sur `ACCESS` . Lorsqu'un *accès* porte les deux à la fois, le *client* visé doit obligatoirement appartenir au *marchand* ciblé (cohérence avec `owns` , règle #2) : contrainte de cohérence entre deux chemins optionnels du modèle, de la même nature que les règles #12 et #16, à vérifier par une contrainte d'intégrité au niveau logique/physique plutôt que par les cardinalités seules. Les cardinalités ne peuvent pas exprimer que dans un couple de valeurs facultatives, celles-ci doivent être mutuellement cohérent dès lors qu'elles sont renseignées.
 25. Entités purement dépendantes (cf. #9) : `CONSENT` (→ `CUSTOMER`) et `DATA_SUBJECT_REQUEST` (→ `CUSTOMER`) ne participent à aucune autre association du modèle (comme `REFUND` , `CHARGEBACK` et `FRAUD_SCORE`). Seule différence avec #9 : le parent peut porter zéro, une ou plusieurs occurrences au fil du temps (i.e. un historique répétable plutôt qu'une extension ponctuelle).
 26. Entités dépendantes à double rôle (cf. #9) : `PRICING_PLAN` (→ `MERCHANT`), `ACCESS` et `CHANGE` (→ `ACTOR`) jouent un second rôle structurel dans le modèle en plus d'être dépendantes pour leur existence : `PRICING_PLAN` est également référencé par `IsRatedUnder` depuis `TRANSACTION` et participe à `IsQuotedIn` vers `CURRENCY` ; `ACCESS` et `CHANGE` portent chacun un lien optionnel supplémentaire vers `MERCHANT` et/ou `CUSTOMER` (`Targets` , `Involves` , `Touche`). Leur dépendance à un parent unique ne les réduit donc pas à de simples extensions isolées.
 27. Suppression des entités dépendantes : la suppression de l'entité parente (`TRANSACTION` , `CUSTOMER` , `MERCHANT` ou `ACTOR`) entraîne logiquement celle de ses entités dépendantes. La règle ne s'applique pas aux entités de référence (`COUNTRY` , `CURRENCY`) ou aux entités à rôles multiples comme `PRODUCT` ou `PAYMENT_METHOD` . Ces entités participent chacune à plusieurs associations et, à ce titre, ne sont pas dépendantes d'un parent unique.
 28. `commission_rate` est un taux, indépendant de toute devise : 2,9 % d'un montant vaut 2,9 % quelle que soit la devise de ce montant, il reste donc un attribut simple de `PRICING_PLAN` . `fixed_fee` , à l'inverse, est une somme fixe (par exemple 0,30 \$ ou 0,25 €) dont la valeur numérique dépend intrinsèquement de la devise : un même plan facture un frais fixe différent selon la devise de la transaction. `IsQuotedIn` porte donc `fixed_fee` comme propriété d'une association *n,n* entre `PRICING_PLAN` et `CURRENCY` , plutôt que de le laisser comme attribut scalaire de `PRICING_PLAN` sous l'hypothèse simplificatrice d'une devise unique. Le frais fixe réellement appliqué à une transaction est celui que porte `IsQuotedIn` pour le couple (plan facturé, devise de règlement de cette transaction - `IsSettledIn`) : encore une contrainte de

cohérence entre deux chemins du modèle, à vérifier au niveau logique/physique. Un plan ne portant pas encore de frais fixe pour une devise donnée ne devrait pas pouvoir facturer une transaction dans cette devise.

29. Une souscription est libellée dans une seule devise (`IsDenominatedIn` , cardinalité 1,1 côté `SUBSCRIBES`) : `amount` (montant prélevé à chaque échéance) n'a de sens qu'accompagné de cette devise. Une même devise peut dénommer 0 à n souscriptions. Cette devise est indépendante de celle des transactions individuellement générées (`Generates` → `IsSettledIn`) : en régime normal les deux coïncident, mais rien dans le modèle ne l'impose structurellement - à traiter comme une contrainte de cohérence supplémentaire si l'hypothèse d'un changement de devise en cours d'abonnement doit être exclue.

1.A.5. Passage au modèle logique

Configuration	Règle	Exemple dans ce modèle
Association $1, n - 1, 1$	L'identifiant du côté $1, n$ migre en clé étrangère vers l'entité du côté $1, 1$	<code>Owns</code> → <code>Customer.merchant_id</code>
Association $0, 1 - 1, 1$	Idem, avec une contrainte d'unicité sur la clé étrangère	<code>Triggers</code> → <code>Refund.transaction_id</code> unique
Association n, n	Devient une table à part entière, portant ses propriétés	<code>Subscribes</code> → table <code>Subscription</code>
Entité de référence	Devient une table ; l'identifiant naturel peut servir de clé primaire	<code>COUNTRY</code> → <code>Country.country_code</code>
Attribut dérivable d'un lien conceptuel	Peut être matérialisé en colonne au MLD/MPD pour des raisons de performance, sans exister comme attribut au MCD	<code>Generates</code> (présence/absence du lien) → <code>Transaction.is_merchant_initiated</code>
Association $1, 1 - 0, n$ portée par une pseudo-entité	La table issue de l'association reçoit directement la clé étrangère (côté $1, 1$)	<code>Authorizes</code> → <code>Subscription.payment_method_id</code>
Propriété portée par une association n, n	Devient directement une colonne de la table issue de l'association	<code>Subscribes.canceled_at</code> → <code>Subscription.canceled_at</code>
Association $0, n - 1, 1$ avec versionage temporel	L'identifiant du côté $0, n$ migre en clé étrangère, accompagné des colonnes de validité déjà portées par l'entité versionnée	<code>BenefitsFrom</code> → <code>PricingPlan.merchant_id</code> (+ <code>effective_from</code> / <code>effective_to</code>)
Association $1, 1 - 0, n$ conservée pour traçabilité malgré une dérivabilité théorique	La clé étrangère est matérialisée explicitement au MLD/MPD, même si elle serait recalculable, pour figer l'historique	<code>IsRatedUnder</code> → <code>Transaction.pricing_plan_id</code>
Valeur dérivée d'une association et des attributs qu'elle relie	Peut être matérialisée en colonne calculée au MLD/MPD pour la performance, sans exister comme attribut au MCD	<code>IsRatedUnder</code> + <code>PRICING_PLAN.commission_rate</code> + <code>IsQuotedIn.fixed_fee</code> → <code>Transaction.fee_amount</code> (OLTP), <code>FactTransaction.fee_amount_eur</code> (OLAP)
Association n, n portant une propriété dépendante des deux entités reliées	Devient une table de jonction à part entière	<code>IsQuotedIn</code> → table <code>PricingPlanFee</code> (<code>pricing_plan_id</code> , <code>currency_code</code> , <code>fixed_fee</code>)
Attribut dérivable via une association tierce, matérialisé pour éviter une jointure	Le lien conceptuel n'existe pas ; l'attribut est dupliqué au MLD	<code>Owns</code> (client → marchand) → <code>ConsentRecord.merchant_id</code> , <code>DataSubjectRequest.merchant_id</code>
Entité conceptuelle sans table dédiée nécessaire	Peut rester un simple attribut texte au MLD si la gestion d'identité est externe au système	<code>ACTOR</code> → <code>DataAccessLog.actor_id</code> / <code>ChangeAuditLog.actor_id</code> (texte, sans table <code>Actor</code>)

1.A.6. Différences entre le modèle de conception et le modèle logique

1.A.6.1. Dans le modèle conceptuel, l'abonnement n'est pas une entité

La relation entre `CUSTOMER` et `PRODUCT` est une association **many-to-many** enrichie d'informations propres. L'entité `Subscription` du modèle logique n'existe pas au niveau conceptuel : un abonnement est conceptuellement une relation entre un client et un produit.

L'abonnement en tant qu'entité viendra avec le modèle logique par application de la règle : toute association n,n devient une table dont la clé primaire est composée des identifiants des entités reliées - ou, choix d'implémentation courant, reçoit un identifiant propre pour simplifier les références.

Subscribes en tant que pseudo-entité conceptuelle.

Le formalisme *Merise* classique ne prévoit pas qu'une association participe directement à une autre association : seules des entités peuvent participer. Ce document déroge à cette règle à deux reprises - Subscribes participe à Generates (vers Transaction) et à Authorizes (vers PaymentMethod).

Ce choix est assumé : dès qu'une association porte des propriétés propres (status, amount...), elle a le statut d'un objet de métier identifiable (un abonnement), et il est plus lisible de la traiter comme telle plutôt que de dupliquer Customer et Product dans deux associations ternaires séparées. Dans le modèle logique cela ne change rien : dans tous les cas Subscribes devient la table Subscription qui porte naturellement les clés étrangères vers Transaction (optionnelle, côté Transaction) et vers PaymentMethod (obligatoire).

Pourquoi une association ternaire Customer - Product - PaymentMethod ne peut pas rempalcer une pseudo-entité?

Parce que PaymentMethod détermine déjà Customer de façon fonctionnelle (cf. règle #3).

Intégrer les trois entités dans une même association ternaire aurait introduit une redondance :

le client deviendrait lisible par deux chemins différents (directement, ou via le moyen de paiement), avec le risque qu'ils divergent.

Passer par Authorizes en associant Subscribes à PaymentMethod évite ce piège (au prix de la règle de gestion #12, qui énonce explicitement la contrainte de cohérence qu'une association simple ne peut pas capturer).

1.A.6.2. Pas de clé étrangère

TRANSACTION ne contient ni merchant_id ni customer_id.

Ces liens sont portés par les associations Receives et Bears.

Les clés étrangères apparaîtront par la règle de transformation :

dans une association $1,n - 1,1$, l'identifiant du côté $1,n$ migre vers l'entité du côté $1,1$.

1.A.6.3. Pas de table de référence d'énumérés

TransactionStatus, ChargebackReasonCode, CardBrand et les autres tables d'énumérés du modèle physique sont des artefacts d'implémentation destinés à garantir l'intégrité et à porter des métadonnées.

Au niveau conceptuel, status est simplement une propriété de TRANSACTION.

1.A.6.4. Les cardinalités portent l'optionnalité

Triggers en $0,1 - 1,1$ énonce la règle métier : une transaction *peut* avoir un remboursement, et un remboursement se rapporte *obligatoirement* à une transaction.

C'est cette cardinalité $0,1$ qui justifie, dans le modèle logique, des tables séparées plutôt que des colonnes optionnelles dans Transaction.

1.A.6.5. Le modèle conceptuel ne distingue pas les deux flux financiers

TRANSACTION porte un amount unique - celui payé par le client.

La répartition entre le **revenu du marchand** (montant moins commission)

et le **revenu de Stripe** (la commission) est une décomposition analytique, pas une propriété conceptuelle de la transaction.

Elle apparaît côté OLAP, où les deux mesures cohabitent sur la même ligne de faits.

Les **paramètres** de calcul de cette commission (commission_rate sur PRICING_PLAN, fixed_fee sur IsQuotedIn) sont bien représentés au niveau conceptuel mais le montant de commission résultant reste une valeur calculée.

1.A.6.6. La commission Stripe ne doit pas être un attribut de TRANSACTION

La commission prélevée sur chaque transaction doit prendre sa source dans le modèle transactionnel. C'est l'entité

PRICING_PLAN et les associations BenefitsFrom et IsRatedUnder qui modélisent les commissions au niveau conceptuel.

Un lien direct est conservé entre TRANSACTION et PRICING_PLAN.

La redondance qui en résulte avec le chemin Receives + BenefitsFrom est

assumée et encadrée par la règle de gestion #16 (exactement comme la redondance `Subscribes / Authorizes` l'est par la règle #12).

`fixed_fee` dépend de la devise.

Le frais fixe prélevé sur chaque transaction est un montant dépendant de la devise associée à la transaction. On a donc une association n,n entre `PRICING_PLAN` et `CURRENCY` : chaque plan peut être réalisé dans plusieurs devises, et chaque devise calcule son prélèvement fixe propre,

sans qu'une conversion de change soit nécessaire. La règle de gestion #28 formalise la contrainte de cohérence qui en résulte.

1.A.6.7. Le volet conformité et sécurité

L'entité `ACTOR` n'a pas de table dédiée dans le modèle logique

Le modèle logique ne comporte pas de table `Actor` (`actor_id` y est un simple attribut texte, référençant une identité gérée ailleurs (SSO, annuaire d'entreprise)).

Le modèle conceptuel introduit malgré tout l'entité `ACTOR`, car conceptuellement, "qui a réalisé cet accès" est un objet métier récurrent au même titre que `CUSTOMER` ou `MERCHANT`, même si, au moment de la transformation en MLD, `actor_id` peut rester une colonne simple sans table de référence propre, si l'équipe juge que la gestion d'identité est du hors périmètre de la base *Stripe*.

`ACCESS` et `CHANGE` sont deux entités distinctes

Dans le modèle conceptuel comme dans le modèle logique, les entités `ACCESS` et `CHANGE` portent de nombreux attributs différents (`source_ip`, `justification`, `was_authorized` pour l'un ; `table_name`, `changed_fields` pour l'autre). La fusion de ces deux entités en une seule table reste utile dans un contexte analytique (reporting consolidé).

Le lien vers `MERCHANT` n'est pas systématique

Les entités `CONSENT` et `DATA_SUBJECT_REQUEST` n'ont pas besoin de ce lien (le marchand est accessible via l'association `Owns` ; voir règle #20).

Les entités `ACCESS`, `CHANGE` et `SECURITY_INCIDENT` portent ce lien explicitement car l'information n'est pas autrement que directement pour ces trois entités (un accès ou un incident peut être transverse à la plateforme, sans qu'il soit possible de l'associer à un marchand).
